# Mysig Book

Everything you need to learn to build a static site with Mysig, including the image-resizing design needed for Zola-style parity.

Mysig is a small, explicit toolkit for building static sites and Gleam web apps without compiler magic.

# Contents

# Mental Model

Mysig is a static-site generator by composition, not by configuration. Your site is a Gleam program. It reads content, renders HTML, copies static files, builds assets, and writes output.

## Core Ideas

- Routes are output paths, normally ending in `index.html` for clean URLs.
- Pages are Lustre element trees rendered to document strings.
- Content is parsed with Pamphlet, which returns metadata and a document body.
- Collections are lists of source files paired with output paths.
- Assets are explicit values declared before rendering code depends on state.
- Production eagerly walks the site, while development can lazily build the same artifacts.

## Why Explicitness Matters

Mysig avoids magic imports such as JavaScript's `import image from "./cat.jpg"`. Instead, a page asks for assets through Mysig APIs, and the runner turns those requests into fingerprinted files.

- Gleam owns the data structures.
- The runner owns filesystem and bundling effects.
- HTML is normal Lustre output.
- A byte change creates a new content-addressed URL.

## What Mysig Does Not Hide

Mysig does not hide that static sites are files on disk. You should know where files come from, where they are written, and what external tools are needed. Image resizing is one of those external tools: Mysig should model requested transforms, but Sharp should do the JPEG, PNG, AVIF, and WebP work.

# Project Setup

Create a JavaScript-target Gleam project and add Mysig plus the libraries used by the current SSG helpers.

```
gleam new my_site
cd my_site
gleam add mysig lustre simplifile filepath snag
```

For Markdown/Djot content, Mysig currently uses Pamphlet from this workspace:

```
pamphlet = { path = "../spotless_server/packages/pamphlet" }
```

For Mysig asset bundling and image resizing on Node, install the Node dependencies used by the runner:

```
npm install --save-dev @rollup/plugin-node-resolve @zip.js/zip.js chokidar rollup
sharp
```

Use these generated directories and ignore them in git:

```
build
public
node_modules
```

## Minimal Main Module

```
import gleam/io
import mysig/ssg
import simplifile

pub fn main() {
  let assert Ok(Nil) = simplifile.create_directory_all("public")
  let assert Ok(files) = ssg.passthrough_files("static", "public")
  let assert Ok(Nil) = ssg.write_files(files, ".")
```

```
  io.println("Built site in public/")
}
```

This copies everything under `static/` into `public/`. From there, add pages and collections.

# Pages And Layouts

Mysig pages are Lustre elements rendered to strings. Keep layouts as functions so all page structure remains typed Gleam code.

```
import lustre/attribute as a
import lustre/element
import lustre/element/html as h

fn shell(title, children) {
  h.html([a.attribute("lang", "en")], [
    h.head([], [
      h.meta([a.attribute("charset", "utf-8")]),
      h.meta([a.name("viewport"), a.content("width=device-width, initial-
scale=1")]),
      h.title([], title),
    ]),
    h.body([], [h.main([], children)]),
  ])
}

fn home() {
  shell("Home", [
    h.h1([], [element.text("Hello Mysig")]),
    h.p([], [element.text("A static page rendered by Gleam.")]),
  ])
  |> element.to_document_string()
}
```

Write the result to `public/index.html` with `simplifile.write` or collect it as an `ssg.File` and pass it to `ssg.write_files`.

## Clean URLs

- `/` maps to `public/index.html`.
- `/about/` maps to `public/about/index.html`.

- `/posts/hello/` maps to `public/posts/hello/index.html`.

Mysig's collection helpers map `about.md` to `about/index.html` and `section/index.md` to `section/index.html`.

# Content Collections

Content collections turn many source files into rendered pages. Mysig uses Pamphlet to parse front matter and Djot content.

```
import filepath
import gleam/list
import lustre/element
import lustre/element/html as h
import mysig/ssg.{type CollectionPage}

fn build_posts() {
  use entries <- result_try(ssg.content_entries("content", "public", ["md"]))
  use files <- result_try(ssg.render_collection(entries, article_layout))
  ssg.write_files(files, ".")
}

fn article_layout(page: CollectionPage(msg)) {
  shell(title(page.metadata), [
    h.article([], [page.content]),
  ])
}

fn title(metadata) {
  case list.key_find(metadata, "title") {
    Ok(title) -> title
    Error(Nil) -> "Untitled"
  }
}
```

## Front Matter

Pamphlet returns metadata as `List(#(String, String))`. Keep metadata lookup at the boundary and convert strings into richer types in your own site code when needed.

- `title`

- description
- date
- draft
- tags
- canonical
- share_image
- share_alt

## Excluding Source Directories

```
let excluded = [".git", "build", "public", "node_modules", "_layouts"]
let assert Ok(paths) = ssg.collect_files_excluding(".", ["md"], excluded)
```

This matters when migrating existing sites with generated output, templates, or project documentation inside the same repository.

# Static Files And Assets

Static files are copied as-is. Assets are declared explicitly and can be fingerprinted by the Mysig asset runner.

## Passthrough Files

```
let assert Ok(files) = ssg.passthrough_files("static", "public")
let assert Ok(Nil) = ssg.write_files(files, ".")
```

Good passthrough files include `robots.txt`, `favicon.ico`, already-built CSS, downloads, videos, and original gallery images when no resizing is needed.

## Fingerprinted Assets

Mysig's asset runner fingerprints bytes with SHA-256 and writes assets under `/assets`. A file such as `logo.png` becomes a content-addressed name such as `logo.<digest>.png`.

Use fingerprinting for CSS generated by a build step, JavaScript bundles, images referenced by components, and resized image variants.

## Client Bundles

```
use cart <- asset.then(asset.bundle("src/cart.gleam", "main"))
```

The server runner builds the bundle and the client runner resolves the generated URL from the manifest injected into the page.

# Data At Build Time

Static sites often need data that is not Markdown. In Mysig, a data source is just a function called by the build program.

## Local Data

```
pub type Product {
  Product(slug: String, name: String, price: Int, summary: String)
}

fn catalog() {
  [Product("linen-bag", "Linen Market Bag", 42, "A strong everyday bag.")]
}
```

## File Data

For JSON, TOML, or CSV, read files at the boundary and decode into typed records before rendering. Keep parsing errors as build failures so broken content cannot deploy silently.

## Remote Data

```
fn fetch_catalog() {
  // Replace the dummy data with an HTTP fetch and decoder.
  Ok(catalog())
}
```

Your rendering code should receive typed data, not raw JSON. This keeps templates simple and makes build failures precise.

# Image Resizing

Image resizing needs an external image engine. Mysig should not implement JPEG, PNG, WebP, AVIF, EXIF, or color-profile handling in Gleam. The right boundary is a typed Gleam request interpreted by a Node runner using Sharp.

## Required Node Dependency

```
npm install --save-dev sharp
```

Sharp provides metadata reading, resize modes, EXIF-aware rotation, JPEG, PNG, WebP, AVIF, quality controls, and progressive JPEG output.

## Required Gleam Model

```
pub type Format {
  Original
  Jpeg
  Png
  Webp
  Avif
}

pub type Fit {
  Fit
  Cover
  Contain
  Inside
}

pub type Resize {
  Resize(
    path: String,
    width: Option(Int),
    height: Option(Int),
```

```
    fit: Fit,
    format: Format,
    quality: Option(Int),
  )
}

pub type Metadata {
  Metadata(width: Int, height: Int, format: String)
}
```

The runner should expose `metadata(path)` and `resize(request)`. Resize returns a Mysig `Asset` whose URL points at a fingerprinted generated image.

## Output Naming

```
<source-path>?w=<width>&h=<height>&fit=<fit>&format=<format>&quality=<quality>
<basename>.<transform-label>.<sha256>.<ext>
```

Hash the generated bytes, not just the request. That preserves the existing Mysig cache rule: if the bytes change, the URL changes.

## Cache Keys

- source file path
- source file digest
- width and height
- fit mode
- output format
- quality
- Sharp version
- Mysig image pipeline version

## Zola-Style Gallery Parity

```
{% set meta = get_image_metadata(path=asset) %}
{% set landscape = (meta.width / meta.height) > 1.2 %}
{{ resize_image(path=asset, width=900, height=900, op='fit') }}
```

The equivalent Mysig flow should request metadata, resize to a 900 by 900 inside-fit thumbnail, and choose `md:w-1/3` for landscape images or `md:w-1/4` otherwise.

```
use meta <- image.then(image.metadata(path))
use thumb <- image.then(image.resize(image.Resize(
  path: path,
  width: Some(900),
  height: Some(900),
  fit: image.Inside,
  format: image.Original,
  quality: None,
)))

let width_class = case meta.width > meta.height * 12 / 10 {
  True -> "md:w-1/3"
  False -> "md:w-1/4"
}

h.img([a.src(asset.src(thumb)), a.class(width_class)])
```

## Responsive Images

A higher-level helper can generate `srcset` variants. Useful widths are 480 for phones, 900 for cards and gallery thumbnails, 1400 for large article images, and 2200 for high-density hero images.

## HTML Rules

- `src` from the chosen fallback variant
- `srcset` for responsive variants
- `sizes` matching the layout
- `width` and `height` from metadata to reduce layout shift

- `alt` from content metadata, never guessed from the filename
- `loading="lazy"` except for the main above-the-fold image
- `decoding="async"` for non-critical images

## Failure Rules

Image errors should fail the build. Do not silently copy originals when a resize fails, because that hides broken deploys and creates performance regressions.

## Implementation Checklist

- Add `sharp` to `package.json` dev dependencies.
- Add a Node FFI module that calls Sharp for metadata and resize.
- Add a Gleam `mysig/image` module with typed requests and results.
- Add a server runner that turns resize requests into fingerprinted `/assets` files.
- Add a dev cache under `build/mysig/images` keyed by source digest and transform.
- Add tests for output names, cache keys, and metadata-based gallery classes.
- Add a migrated gallery example that proves Zola `resize_image(... op='fit')` parity.

# Development, Export, And Deploy

Mysig's development and production model should produce the same artifacts. The difference is when work happens.

## Development

- render the requested route
- build only assets needed by that route
- watch source files and restart quickly
- inject a manifest so client asset lookup matches production

## Static Export

- render every HTML page
- copy passthrough files
- build client bundles
- resize all requested image variants
- write fingerprinted assets
- fail on any missing content, asset, or image transform

## Verification

```
gleam format --check
gleam test
gleam run
```

For migrations, compare old and new output: route paths, canonical URLs, static assets, image dimensions and formats, generated HTML for critical pages, and mobile and desktop screenshots.

## Deployment

The deploy target is the generated `public/` directory. Configure hosting to serve fingerprinted assets with long-lived immutable cache headers and HTML with short cache headers.

Useful feedback measures include time from fresh clone to first successful build, number of route differences during migration, image weight before and after resizing, Largest Contentful Paint on pages with hero images, and broken-link count after export.